

Enterprise Hybrid RAG

Design and construction report

Document-grounded question answering over PDF, DOCX and Markdown,
built without a retrieval framework

Tú Nguyễn

September 15, 2026

Abstract. This report describes the design, construction and evaluation of a retrieval-augmented question answering system built from primitives rather than from a framework, so that every ranking decision is inspectable and every published number is reproducible. The system fuses dense (bi-encoder) and sparse (BM25) retrieval with Reciprocal Rank Fusion and reorders the result with a cross-encoder, then answers only from the retrieved text and abstains when retrieval confidence is too low.

The central empirical result is that the components are not independently good. Replacing the offline fallback backends with real neural models makes dense retrieval on its own *worse* — Recall@1 falls from 0.6550 to 0.5853 — while the full fused-and-reranked pipeline improves by 0.1512. The reason is visible in a per-question-type breakdown: lexical matching is perfect on reference codes and nearly useless on paraphrase; a sentence encoder is the reverse. The value of hybrid retrieval is not an average of the two but the recovery of both ceilings at once, and that recovery requires the cross-encoder — plain fusion scores *worse* than either retriever on the category each one owns.

A second experiment grows the corpus from 77 to 8548 chunks with in-domain distractors while holding the questions fixed. Dense retrieval loses 35% of its Recall@1; the reranked pipeline loses 9% and then stops, holding 0.7248 across a twentyfold growth in the haystack. The reranker’s advantage over dense retrieval widens from +0.2093 to +0.3450: it does not merely survive scale, it is worth more at scale.

The report also describes the measurement discipline that makes those numbers worth reading — guards that abort a run rather than publish a misleading figure — and three occasions on which that discipline caught the author’s own mistakes, including a continuous-integration check that reported success while discarding the result it was meant to verify.

Repository: <https://github.com/tu-h-nguyn/Enterprise-Hybrid-RAG>

All figures in this report are drawn from committed evaluation artefacts.

Contents

1	The problem, and the shape of the answer	2
1.1	Scope	2
2	Architecture	2
2.1	Backends degrade, and say so	3
3	Design decisions worth defending	3
3.1	Reciprocal Rank Fusion, not score averaging	3
3.2	Ground truth is answer spans, not chunk identifiers	3
3.3	Confidence thresholds are per stage, because the scales differ	4
3.4	The user interface is a client, not a second implementation	4
4	Evaluation methodology	4
4.1	The dataset	4
4.2	What makes a number worth publishing	4
4.3	Where the numbers come from	5
5	Results	5
5.1	Retrieval comparison	5
5.2	The components are not independently good	5
5.3	A metric read against the wrong ceiling	6
5.4	Does it survive a bigger corpus?	7
5.5	Chunk size, and changing a default on evidence	7
5.6	What approximate search costs	8
5.7	Cost	8
5.8	Generation	8
6	How the system is used	9
6.1	Three surfaces, one implementation	9
6.2	Getting it running	10
6.3	Choosing a generation backend	10
6.4	Where it fits, and where it does not	10
7	Engineering process	10
7.1	What continuous integration actually enforces	10
7.2	Three times the discipline caught the author	11
8	Limitations	12
9	What comes next, in order	12
A	Repository map	13
B	Reproducing every figure in this report	13

1 The problem, and the shape of the answer

An organisation holds its operating knowledge in documents: a handbook, a security policy, an incident runbook, a retention schedule. People do not want to read them; they want to ask them a question. The failure mode of a language model asked such a question directly is well known — it produces a fluent answer with no source, and the answer is sometimes wrong in a way the reader cannot detect.

The system described here constrains the answer to retrieved text, attaches a citation to every claim, and refuses when retrieval is not confident enough. The pipeline is deliberately ordinary:

```
ingest → chunk → dense + BM25 index → RRF fusion → cross-encoder rerank → context build  
→ grounded generation
```

What is not ordinary is the decision to build it without a retrieval framework. The core retrieval path has no LangChain, no LlamaIndex, no abstraction that hides a ranking decision behind a configuration flag. That choice cost time and bought two things that turned out to matter more than the time: every scoring decision is a function that can be read in a minute and unit-tested against a hand-computed example, and every number in the evaluation can be traced to the code that produced it.

1.1 Scope

- **In scope.** Loading PDF, DOCX and Markdown with page and section provenance; chunking that preserves that provenance; two independent retrievers and an explicit fusion rule; reranking; an abstention gate; citation resolution; a FastAPI service; a Streamlit interface; an evaluation harness with a labelled dataset; continuous integration that regenerates the published numbers rather than trusting them.
- **Out of scope.** Multi-hop question decomposition, incremental index updates, authentication and multi-tenancy, and fine-tuning any model. Each is named in the limitations rather than quietly omitted.

2 Architecture

The system is a library with three entry points — an HTTP API, a set of command line scripts, and an evaluation harness — all of which call the same method. That is a deliberate constraint rather than an accident of layering: if the evaluation called a reimplement of retrieval, the published numbers would describe code that no user ever runs.

Layer	Responsibility
app/ingestion	Loaders (PyMuPDF, python-docx, Markdown), cleaning, and a metadata-aware chunker that carries page number and section heading onto every chunk. Provenance is attached at load time and never re-derived, which is what makes a citation trustworthy.
app/indexing	Embedding backends, a Chroma or NumPy vector store, and a persistent BM25 index. Each backend reports what it actually is, not what was requested.
app/retrieval	Dense retrieval, sparse retrieval, Reciprocal Rank Fusion, rerankers, and a context builder that deduplicates and enforces a token budget.
app/generation	Provider adapters (OpenAI-compatible, Anthropic, offline extractive), prompt construction, and citation parsing that discards references the model invented.
app/services	RagService — the single entry point — plus the abstention gate and document lifecycle.
app/evaluation	Dataset schema, relevance resolver, retrieval and generation metrics, and the experiment runners.

2.1 Backends degrade, and say so

Every model-backed component can be set to `auto`. If the model hub is unreachable — which is the normal condition inside a restricted build environment — the embedder falls back to a TF-IDF plus truncated-SVD encoder, the reranker falls back to a feature-based scorer, and generation falls back to extractive sentence selection.

The important part is the second half of that sentence. Degrading silently would be a liability: a benchmark run on fallbacks would publish numbers under a name that implies neural models. So every artefact records which backend produced it, `/health` reports the same, the report renderer emits an explicit warning block, and the continuous integration job that publishes neural numbers *asserts* a 384-dimension sentence-transformer in the index manifest before it measures anything.

3 Design decisions worth defending

3.1 Reciprocal Rank Fusion, not score averaging

Cosine similarity lives in $[-1, 1]$. BM25 is unbounded and corpus-dependent. Averaging or weighting them makes the blend depend on one query’s score distribution, so a single outlier BM25 hit dominates the result. Reciprocal Rank Fusion discards magnitudes and uses ranks only:

$$\text{RRF}(d) = \sum_i \frac{w_i}{k + \text{rank}_i(d)}, \quad k = 60.$$

With $k = 60$, a document ranked first by one retriever cannot beat a document ranked second or third by both — agreement between two independent retrievers is the signal being bought, and the constant is what makes it so. The implementation is roughly fifteen lines and is unit-tested against a hand-computed example, so the constant’s effect is checkable rather than folklore.

3.2 Ground truth is answer spans, not chunk identifiers

The evaluation dataset labels each question with its source documents and a verbatim answer span, and resolves those to chunk identifiers at evaluation time.

Pinning chunk identifiers would have been easier and would have quietly destroyed the most useful experiment in the project. Chunk identifiers depend on the chunking configuration; the moment the chunk size changes, pinned labels address nothing, and every metric collapses to zero

in a way that looks exactly like a retrieval failure. Because labels are spans, the chunk-size sweep in Section 5.5 is possible at all — and, as measured in that section, the resolved gold sets are identical in size and shape at both chunk sizes, which is what makes the comparison fair.

3.3 Confidence thresholds are per stage, because the scales differ

Cosine similarity is in $[-1, 1]$; fused RRF scores are around $[0, 0.03]$; BM25 is unbounded; cross-encoder logits run roughly $[-11, +11]$; the lexical fallback is around $[0, 5]$. A single “confidence” threshold across those scales is not a tuning problem, it is a category error. Each stage therefore owns its own threshold, and the abstention response records which one fired.

3.4 The user interface is a client, not a second implementation

The Streamlit interface holds no retrieval logic. It speaks HTTP to the same five endpoints an external client would use, so what a visitor sees is what the API serves.

That constraint was tested when the project needed to run on a single-process host — Streamlit Community Cloud and Hugging Face Spaces give you one process, with nowhere to put a separate server. The obvious solution was to let the interface import the service and call it directly. It was rejected for the reason above. Instead the real ASGI application is started on a loopback socket inside the same process and the interface keeps speaking HTTP to it. The transport changes; the contract does not. A test asserts that every endpoint the interface calls exists on the application that gets embedded, so the demonstration cannot drift away from the deployment the documentation describes.

4 Evaluation methodology

4.1 The dataset

Fifty questions over twenty-two documents, written by hand against the corpus and typed by the failure they are meant to expose:

Type	n	What it tests
factual	14	A fact stated once, in the words the question uses.
keyword	7	A reference code — HB-7.2, FIN-TE-4, SEV1. Lexical matching should win.
paraphrased	9	The answer exists but shares no vocabulary with the question. Semantic matching should win.
multi_step	7	Evidence from two or more documents.
terminology	6	A defined term asked plainly.
unanswerable	7	Nothing in the corpus answers it. The system should refuse.

The seven unanswerable questions are not filler. Without them, an abstention gate can be tuned to never abstain and the metrics improve.

4.2 What makes a number worth publishing

Three failures would each turn this evaluation into a confident lie, so each one aborts the run rather than reporting a figure:

1. **A distractor that restates a labelled answer.** Relevance resolves per source document, so a distractor can never be *labelled* relevant — but it would be a correct answer scored as a miss. The corpus-scale run refuses to start if any distractor chunk contains a labelled span.

2. **Gold chunks that move as the corpus grows.** If re-ingestion shifted the resolved gold set, a fall in Recall@1 would be a labelling artefact read as a scale effect. The run asserts the set is identical at every size — per chunk size, because spans legitimately resolve to different chunks when the chunking changes.
3. **An embedder whose width changes with the corpus.** The offline TF-IDF fallback derives its dimensionality from the corpus rank, so a sweep on it varies the encoder along with the variable under study. Continuous integration asserts a fixed 384 dimensions at every size before it will publish anything.

A fourth discipline is procedural rather than assertive: quality metrics are deterministic given the same corpus, dataset and backends, so a re-run is a reproducibility test. The comparison script projects a result file down to its deterministic parts, excludes wall-clock timings, and refuses outright to compare runs made on different backends. Identical output means nothing drifted; a difference is a real change worth committing.

4.3 Where the numbers come from

The development environment could not reach the model hub, so every neural figure in this report was produced by a GitHub Actions workflow that pins the neural backends explicitly rather than using `auto`. Under `auto`, an unreachable hub would fall back silently and publish fallback numbers under a workflow named “neural” — precisely the failure the project is built to avoid.

5 Results

Unless stated otherwise: `all-MiniLM-L6-v2` as the bi-encoder, `ms-marco-MiniLM-L-6-v2` as the cross-encoder, 22 documents and 77 chunks at the shipped 256-token chunk size, 50 questions of which 43 are answerable.

5.1 Retrieval comparison

Configuration	R@1	R@3	R@5	R@10	MRR	nDCG@5	ms
Dense only	0.5853	0.8256	0.8953	0.9302	0.7740	0.7869	16.58
BM25 only	0.6434	0.8023	0.8023	0.8023	0.7597	0.7660	0.78
Hybrid (RRF)	0.6434	0.8023	0.8488	1.0000	0.7910	0.7862	17.04
Hybrid + Reranker	0.7946	0.8953	0.9070	1.0000	0.9085	0.8937	699.95

Two columns carry more information than the headline. BM25’s recall is *identical* at ranks 3, 5 and 10 — it finds the chunk in the first three results or it never finds it, which is what a lexical matcher does when the query’s words are not in the text. And both fused configurations reach 1.0000 at rank 10: everything this corpus can answer is inside ten candidates, so from there the problem is entirely ordering. That is why a reranker can reach 0.7946 at rank 1 without a better retriever underneath it.

5.2 The components are not independently good

Configuration	R@1 fallback	R@1 neural	MRR fallback	MRR neural
Dense only	0.6550	0.5853	0.7785	0.7740
BM25 only	0.6434	0.6434	0.7597	0.7597
Hybrid (RRF)	0.6550	0.6434	0.7784	0.7910
Hybrid + Reranker	0.6434	0.7946	0.7541	0.9085

A better embedder made dense retrieval on its own *worse*, left BM25 exactly where it was — as it must, since BM25 does not use the embedder, which is a useful check that nothing else drifted between the two runs — and moved plain fusion by less than a question. Only the full pipeline gained, and it gained +0.1512 Recall@1 and +0.1544 MRR.

The explanation is entirely in the breakdown by question type.

Recall@1	TF-IDF	Dense (MiniLM)	BM25	Hybrid + Rerank	ceiling
keyword	1.0000	0.4286	1.0000	1.0000	1.0000
paraphrased	0.1111	0.4444	0.1111	0.4444	1.0000
terminology	0.8333	0.3333	0.8333	1.0000	1.0000
factual	0.8571	0.9286	0.8571	1.0000	1.0000
multi_step	0.4524	0.4524	0.3810	0.4524	0.4524
all answerable	0.6434	0.5853	0.6434	0.7946	0.9109

The TF-IDF fallback is perfect on keyword questions, because character n-grams match a reference code literally, and nearly useless on paraphrase. MiniLM is the mirror image. A sentence encoder compresses FIN-TE-4 into a dense vector and loses it.

The result the project exists to demonstrate

Neither retriever is better. They fail in opposite directions. Fused and reranked, the pipeline holds *both* ceilings at once — keyword 1.0000 and paraphrased 0.4444, each matching the best single retriever — and beats both of them on terminology, 1.0000 against 0.8333. Its MRR is a perfect 1.0000 on four of the five answerable categories.

And fusion alone does not get there. Plain RRF scores 0.5714 on keyword and 0.2222 on paraphrased: *worse than BM25 and worse than dense respectively*, on the category each of them owns. Averaging two rankings that disagree lands between them. The cross-encoder, which reads query and chunk jointly rather than comparing two independently compressed vectors, is what turns a fused candidate list back into the best of both.

5.3 A metric read against the wrong ceiling

Recall@ k is bounded by $\min(k, |R|)/|R|$, where R is the set of relevant chunks. A question with two relevant chunks cannot score above 0.5000 at $k = 1$; one with three cannot exceed 0.3333. Every `multi_step` question in this dataset has two or three, which puts the ceiling for that column at exactly 0.4524.

The reranked pipeline scores exactly 0.4524, with Recall@5 of 1.0000 and MRR of 1.0000.

The weakest-looking category was the most solved one

`multi_step` is not the weakest answerable category. It is saturated — every relevant chunk for every multi-step question is retrieved, with one of them always at rank 1. This project’s own documentation called it “the weakest answerable category” and “the honest next problem” in four separate places, because 0.4524 was read against an imagined 1.0 instead of against the bound the labels impose. The evaluator now reports `max_recall@k` beside `recall@k`, so the number cannot be misread the same way again.

Correcting the bound changes the overall reading as well. Across all 43 answerable questions the achievable Recall@1 is **0.9109**, not 1.0, so the reranked pipeline’s 0.7946 is **87.2%** of what any retriever could reach on this dataset. The genuinely open category is `paraphrased`, at 0.4444 against a ceiling of 1.0000 — and unlike `multi_step`, that gap is real.

5.4 Does it survive a bigger corpus?

Seventy-seven chunks is a haybale, not a haystack: forty of them are the answer to some question, and the top five cover 6.5% of everything there is. Recall@5 saturates, and none of it answers whether the ranking survives a realistic index.

So the fifty questions, their answer spans, the chunking and every retrieval setting were pinned, and only the corpus grew — with distractor documents on the *same topics, in the same register, with the same policy vocabulary*, for other fictional companies. Padding with off-topic prose would have proved nothing: a policy question never retrieves a novel, recall would have stayed flat, and the result would have been a rigged win.

Chunks	Docs	Gold share	Dense	BM25	Hybrid	Hybrid + Rerank
77	22	51.95%	0.5853	0.6434	0.6434	0.7946
423	162	9.46%	0.4690	0.6085	0.5853	0.7248
1704	677	2.35%	0.4341	0.5969	0.5620	0.7248
8548	3422	0.47%	0.3798	0.5969	0.4922	0.7248
Change over 111×			−0.2055	−0.0465	−0.1512	−0.0698

The reranker is worth more at scale, not less

Dense retrieval gives up 35% of its Recall@1 across the range. The reranked pipeline gives up 9% and then *stops*: 0.7248 at 423 chunks, 0.7248 at 1704, 0.7248 at 8548 — flat across a twentyfold growth, while the gold chunk goes from one in 1.9 to one in 213. Its advantage over dense alone widens from +0.2093 to +0.3450. That is the argument for paying its latency, and it is completely invisible at the size the rest of the evaluation runs at.

BM25 is nearly flat (−0.0465, two questions), which is the expected shape: exact lexical matching does not care how much other text exists, only whether something else matches better.

Are the distractors actually hard? Asserting it would be worthless, so every row measures it. At 8548 chunks, a distractor takes rank 1 on 41.9% of answerable questions under dense retrieval and distractors hold 75.4% of the top five. They compete. The reranked pipeline holds its own rate at a flat 0.1628 from 423 chunks upward — the same shape as its Recall@1.

5.5 Chunk size, and changing a default on evidence

The chunk-size ablation had said for some time that 256 tokens beat the shipped 500 by 9.3 points of Recall@1 for the production configuration. The default was left alone, and the report said why: one fifty-question corpus of 44 chunks is thin evidence, and a chunk size that wins on a corpus that small may be a fact about the corpus rather than about chunking.

That objection was answerable, so it was answered. The scale sweep runs the whole ladder at both sizes, matched by how many distractor *documents* were added, because the same documents become a different number of chunks at each size.

Distractor docs	256 tokens			500 tokens		
	chunks	R@1	MRR	chunks	R@1	MRR
0	77	0.7946	0.9085	44	0.7016	0.8568
140	423	0.7248	0.8384	235	0.6550	0.8101
655	1704	0.7248	0.8289	936	0.6318	0.7950
3400	8548	0.7248	0.8285	4658	0.6318	0.7868

256 wins Recall@1 at every size, by 0.0930 at the largest, and the gap does not shrink as the corpus grows. It also halves the cost of the dominant stage: 685.39 ms against 1370.06 ms in the same sweep, because the cross-encoder scores passages half as long. These rows then reproduced exactly on a second runner, with the roles of default and alternative swapped between the two runs. The default moved to 256 on that evidence.

The cost, stated rather than buried. 500 wins Recall@5, 0.8566 against 0.8062 at the largest corpus. Two things decide it anyway. First, the resolved gold sets are identical at both chunk sizes — mean 1.209 relevant chunks per question, distribution {1:36, 2:5, 3:2} — so this is not a labelling artefact; that was checked, not assumed. Second, Recall@5 is not a like-for-like comparison across chunk sizes: five chunks of 500 tokens hand the generator twice the text that five chunks of 256 do. The metrics where a longer chunk is *structurally* advantaged are Recall@1 and MRR — a bigger chunk is a bigger target to hit — and 256 wins those anyway, by nine points. The metric 500 wins is the one where it is given twice as much room. A comparison at a fixed context budget, top_k = 10 at 256 against top_k = 5 at 500, is recorded as the follow-up this does not attempt.

5.6 What approximate search costs

Every size was also run against both vector stores — Chroma’s HNSW index and exhaustive cosine — because that difference is normally assumed rather than measured. With all-MiniLM-L6-v2, Recall@1 is identical for every configuration at every size, and the single difference anywhere in the sweep is 0.0023 of dense MRR at the largest corpus. BM25 never touches the vector store and reads identical everywhere, which is the control confirming nothing else differed between the two runs.

That result does not transfer to any embedding. Under the offline TF-IDF fallback the same comparison loses 0.0233 *Recall@1* to HNSW and is not reproducible at all: repeated runs of the same command returned dense Recall@1 of 0.3643, 0.3411, 0.3876 and 0.3876 — the last two with BLAS threading pinned to one core, which ruled out floating-point reduction order and pointed at the index itself. Exhaustive search returned 0.4109 twice and matched on all 86 compared values. “HNSW is fine” is a statement about how well separated these embeddings are, not about HNSW.

5.7 Cost

	77 chunks	8548 chunks
Index build	~9 s	~3.7 min
Dense query (exhaustive)	~15 ms	~21 ms
BM25 query	under 1 ms	~16 ms
Hybrid + rerank query	~0.7 s	~0.7 s

Rounded on purpose: these are wall-clock figures on a shared runner and move by tens of percent between runners. The shape is the claim. Reranking is flat because it always rescores a fixed top-*k* — the cross-encoder never sees the corpus — and it is 98% of query latency at 77 chunks and still 94% at 8548. Growing the corpus by 111× does not change what dominates a request; what it changes is the index build, which is where the corpus is actually paid for.

5.8 Generation

Generation is measured on the *extractive* backend, because continuous integration has no API key. These numbers describe sentence selection, not a language model, and are reported for completeness rather than as a claim about answer quality.

Metric	Value
groundedness (answer bigrams present in context)	0.6434
answer_f1 (SQuAD-style token F1)	0.3137
span_coverage (labelled span appears in answer)	0.5078
citation precision	0.6512
correct refusal (unanswerable)	0.8571
over-refusal (answerable)	0.3488

The gate correctly refuses six of seven unanswerable questions and also refuses fifteen of forty-three answerable ones. Both rates are unchanged from the fallback run and did not move when the chunk size changed, which is itself informative: the gate’s behaviour here is dominated by the extractive backend’s exact-token matching rather than by retrieval quality. Measuring it properly requires a real language model, and that is the first open item.

6 How the system is used

6.1 Three surfaces, one implementation

As an HTTP service. The intended production shape. `POST /query` takes a question and returns an answer, citations resolved to document, page and section, the retrieved chunks, and a trace of what each stage did and how long it took. `GET /health` reports index readiness and — importantly — which backends were actually selected, so a degraded deployment is visible from outside. `POST /documents/upload` and `POST /documents/index` cover the ingestion lifecycle.

As a user interface. A Streamlit application for the people who will not write `curl`. It shows the answer, every citation as an expandable block with its snippet, the retrieved chunks with their component scores, and a debug panel containing the raw trace. When the abstention gate fires, it shows which threshold fired rather than a generic apology.

With `API_URL` pointing at a running service, it is a thin client. With `API_URL` unset, it starts the service inside its own process and builds the index on first visit — which is what makes it deployable to a single-process host with no configuration at all.

As a library and a set of scripts. `scripts/ingest.py` builds the indexes; `scripts/evaluate.py` scores retrieval or generation; `scripts/benchmark.py` runs the full comparison and ablation and writes `results.json` and `report.md`; `scripts/scale_experiment.py` runs the corpus-scale sweep; `scripts/compare_results.py` decides whether two runs agree on their deterministic parts.

6.2 Getting it running

Goal	Command
Local service and interface	<pre> pip install -r requirements.txt python scripts/ingest.py uvicorn app.main:app -reload API_URL=http://localhost:8000 streamlit run frontend/streamlit_app.py </pre>
Everything in one process	<pre> streamlit run frontend/streamlit_app.py </pre> (no <code>API_URL</code>: the service starts in-process and the index is built on first visit)
Containers	<pre> docker compose up -build </pre> (API on 8000, interface on 8501)
Reproduce the numbers	<pre> python scripts/benchmark.py python scripts/scale_experiment.py -extra-chunk-sizes 500 </pre>

6.3 Choosing a generation backend

The embedder and the reranker are public models and need no token of any kind. Only generation needs a key, and only if a generative answer is wanted:

- **Nothing configured** — the extractive backend selects supporting sentences from the retrieved context. No key, no network, and every artefact says so.
- **A local model server** — point `LLM_BASE_URL` at an OpenAI-compatible endpoint such as Ollama. No key leaves the machine and no document does either. Two traps are documented because they cost time: `LLM_API_KEY` must be non-empty even though such a server ignores it, and `localhost` inside a container must become `host.docker.internal`.
- **A hosted provider** — OpenAI-compatible or Anthropic, selected by environment variable. Both adapters are covered by tests that stand up a real HTTP server on a loopback port and assert what each one actually sends: the endpoint, the authentication header, and the request shape.

No secret is ever written to a file in the repository, baked into an image, or printed in a log.

6.4 Where it fits, and where it does not

The system suits an internal corpus of policy-like documents — handbooks, runbooks, contracts, standards — where answers must be attributable and a confident wrong answer is worse than an admission of ignorance. It does not suit open-domain question answering, questions requiring synthesis across many documents (see `multi_step` in the results), or a corpus that changes continuously, because indexing is a full rebuild rather than an incremental update.

7 Engineering process

7.1 What continuous integration actually enforces

- **Lint, types and tests** — `ruff`, `mypy` over `app/`, `scripts/` and `frontend/`, and 151 tests behind a coverage floor that fails the build if it drops.
- **A blocking dataset audit**. Evaluation labels are resolved against freshly ingested chunks; if any label stops resolving, the build fails. This gate was verified by deliberately corrupting an

answer span and confirming a non-zero exit — a gate nobody has seen fail is a gate nobody should trust.

- **A container that is built, ingested into, served and queried.** Not built and declared working.
- **An image-size ceiling and a +cpu assertion,** because torch arrives as a transitive dependency and the Linux wheel on the public index is the CUDA build — roughly 3 GB of wheels for a container with no GPU.
- **Regenerated benchmarks.** The neural numbers are recomputed and compared against the committed ones on quality metrics only, since latency is wall-clock. Identical output means the benchmark reproduced; it commits a new artefact only when a quality metric actually moved.

7.2 Three times the discipline caught the author

A report that lists only successes is not evidence of engineering judgement. The following are the three most useful failures of this project, all of them found by machinery rather than by inspection.

1. A benchmark that had never run

The retrieval evaluator unpacked `retrieve()` as `_, final, _` when the method returns `(candidates, trace, stopwatch)` — it was iterating a trace object. The phase was marked complete in the handover document. It had never been executed end to end. The fix was one line; the lesson was that “done” in a status table is a claim, not a measurement.

2. Numbers transcribed into prose go stale

Wall-clock figures were copied into the documentation — 1416.77 ms, 154.99 s — and a later run on a different runner invalidated all of them. The mistake was the transcription, not the figures: shared-runner timings move by tens of percent and say nothing the shape does not. They are now rounded, labelled as one run’s wall-clock, and pointed at the committed artefact. A script checks that every four-decimal number in both READMEs traces back to a committed artefact; the only figures that do not are a Python version, an image size explicitly labelled as the *before*, and an example payload labelled with the default it was captured at.

3. A check that reported success while it was wrong

Chroma’s HNSW index is approximate, and two identical runs disagreed on a handful of deep-rank values. To stop that rewriting the artefact on every run, the comparison script gained an `-allow-drift` prefix. Then the chunk size changed, which swapped which sizes each store runs at — so the regenerated artefact carried `chroma/c256/...` rows where the committed one had `chroma/c500/...`. Every one of those keys existed on exactly one side, every one began with `scale.chroma/`, and the guard counted them all as tolerable noise. The job printed *“reproduced the committed numbers exactly; nothing to commit”* and discarded a correct measurement.

Drift now means a value moved. A key present on one side and absent on the other is a structural change and no prefix excuses it. The test that reproduces the original failure fails without the fix.

The third is the one worth dwelling on. The guard was added for a real reason and made the system quieter; it also made it lie, and it lied in the direction of reporting success. A check that can only fail loudly is worth more than a check that is usually right.

8 Limitations

Each of these bounds how far the numbers above generalise.

1. **The corpus is synthetic, and the labelled part of it is small.** Twenty-two documents, 77 chunks, 43 scored questions — where one question is 0.0233. The per-type rows rest on six to fourteen questions each. The scale experiment grows the index to 8548 chunks, but with generated documents: hard negatives by construction, where a real corpus that size would hold both easier negatives and harder ones, such as near-duplicate revisions of the same policy. Treat every comparison as directional.
2. **Generation is not measured with a real language model.** The retrieval numbers are real; the generation numbers describe sentence selection. Everything about the abstention gate is therefore provisional.
3. **The abstention threshold is tuned on one corpus** and over-refuses 34.88% of answerable questions. That is the system's worst product-level flaw, and it is downstream of the previous item.
4. **Token counts are a chars/4 heuristic**, not a real tokenizer, so chunk sizes and context budgets are approximate and a real tokenizer would shift chunk boundaries.
5. **Reranking dominates latency** — 699.95 ms against BM25's 0.78 ms. Any deployment must decide whether that precision is worth nearly three orders of magnitude, or whether to rerank only when the fusion margin is narrow.
6. **Indexing is a full rebuild**, not an incremental upsert. Correct at this size, wrong at scale.
7. **paraphrased is the weakest answerable category** (0.4444 Recall@1 against a ceiling of 1.0000). Vocabulary mismatch between question and document is the one failure neither retriever handles well and reranking only partly repairs.

9 What comes next, in order

1. **Measure generation against a real model.** A local server needs no key and keeps the corpus on the machine, and it unlocks the judge that refuses to run on the extractive backend. Items 2 and 3 are blocked on this, directly or indirectly.
2. **Compare chunk sizes at a fixed context budget** ($\text{top_k} = 10$ at 256 against $\text{top_k} = 5$ at 500), which is the one comparison the chunk-size decision did not control for.
3. **Re-tune the abstention thresholds** once there is a real generator to tune them against.
4. **Attack paraphrased** — now that the recall ceiling is computed, it is the only answerable category with real headroom.
5. **Incremental indexing**, so a changed document does not require a full rebuild.

A Repository map

Path	
app/ingestion/	Loaders, cleaner, metadata-aware chunker
app/indexing/	Embedders, Chroma and NumPy vector stores, BM25
app/retrieval/	Dense, sparse, RRF fusion, rerankers, context builder
app/generation/	Providers, prompts, citation resolution
app/evaluation/	Dataset schema, resolver, metrics, experiment runners
app/services/	RAG pipeline, abstention gate, document lifecycle
frontend/	Streamlit interface and the in-process API starter
scripts/	Ingest, evaluate, benchmark, corpus-scale experiment
tests/	151 unit and integration tests
data/evaluation/	The labelled dataset and every committed result artefact
HANDOFF.md	Interface contracts, status by phase, and what is open

B Reproducing every figure in this report

1. `pip install -r requirements.txt`
2. `python scripts/make_demo_corpus.py` — writes the 22 documents
3. `python scripts/ingest.py` — 22 documents, 77 chunks
4. `python scripts/benchmark.py` — the comparison, the ablation and generation; writes `data/evaluation/results`
5. `python scripts/scale_experiment.py -extra-chunk-sizes 500` — the corpus-scale sweep; writes `scale_results.json`

Without access to the model hub, steps 4 and 5 will run on the offline fallbacks and every artefact they write will say so in its `backends` block and in the warning at the top of its report. The neural figures in this document come from the `benchmark.yml` workflow, which pins the backends and asserts a 384-dimension embedder before measuring.

Every figure in this report was taken from a committed evaluation artefact produced by a run that actually executed. Where a number is historical — a previous image size, an example payload captured at an earlier default — it is labelled as such in the text.